

Foundation Models and Generative Artificial Intelligence

ARIN 5203

Instructor: Dr. Qifeng Chen

Deep Generative Models

Deep Generative Models

Learning to generate data

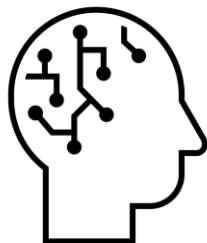


Samples from a Data Distribution

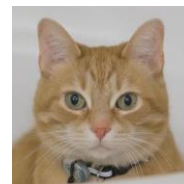
Train



Neural Network

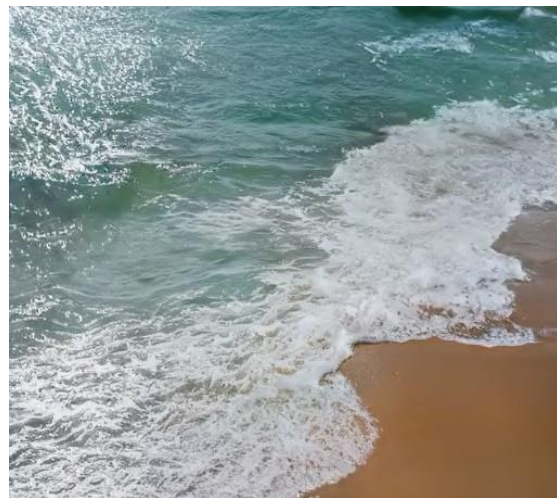


Sample



Application (1): Content Generation

StyleGAN3 example images



Application (2): Representation Learning

Learning from limited labels



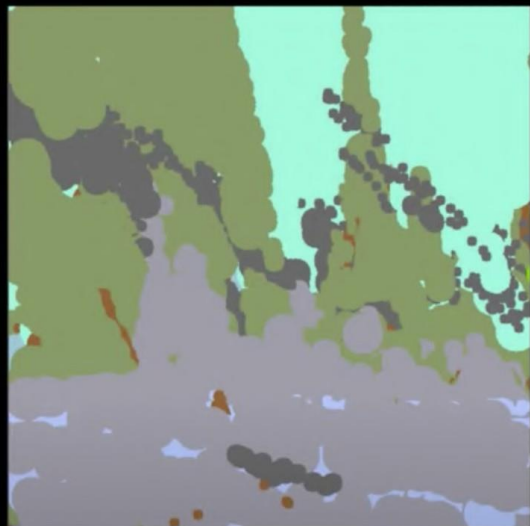
[Zhang et al., DatasetGAN: Efficient Labeled Data Factory with Minimal Human Effort, CVPR 2021](#)

[Li et al., Semantic Segmentation with Generative Models: Semi-Supervised Learning and Strong Out-of-Domain Generalization, CVPR 2021](#)

slide from <https://cvpr2022-tutorial-diffusion-models.github.io/>

Application (3): Artistic Tools

NVIDIA GauGAN



NVIDIA GauGAN Segmentation Input



NVIDIA GauGAN Output

2022 / 2023 : The year of generative modeling?



DALL·E 2

Stable Diffusion

Where we came from

VAEs, 2013



GANs, 2014



PixelCNN, 2016



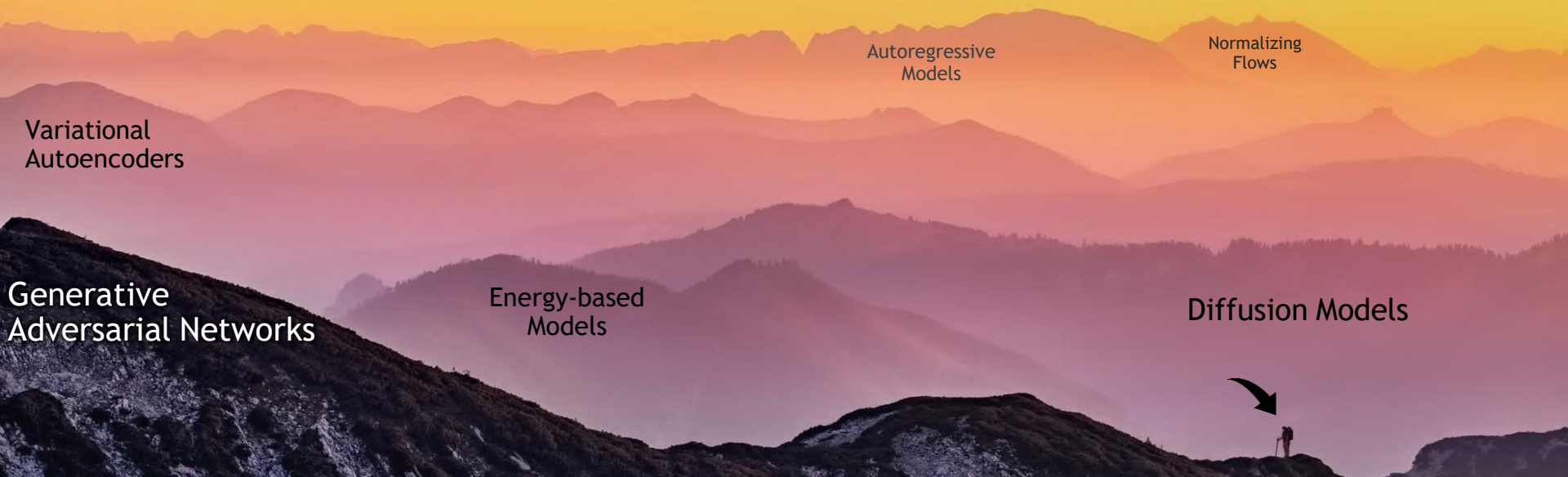
BigGAN, 2019



Imagen, 2022



The Landscape of Deep Generative Models



Variational
Autoencoders

Autoregressive
Models

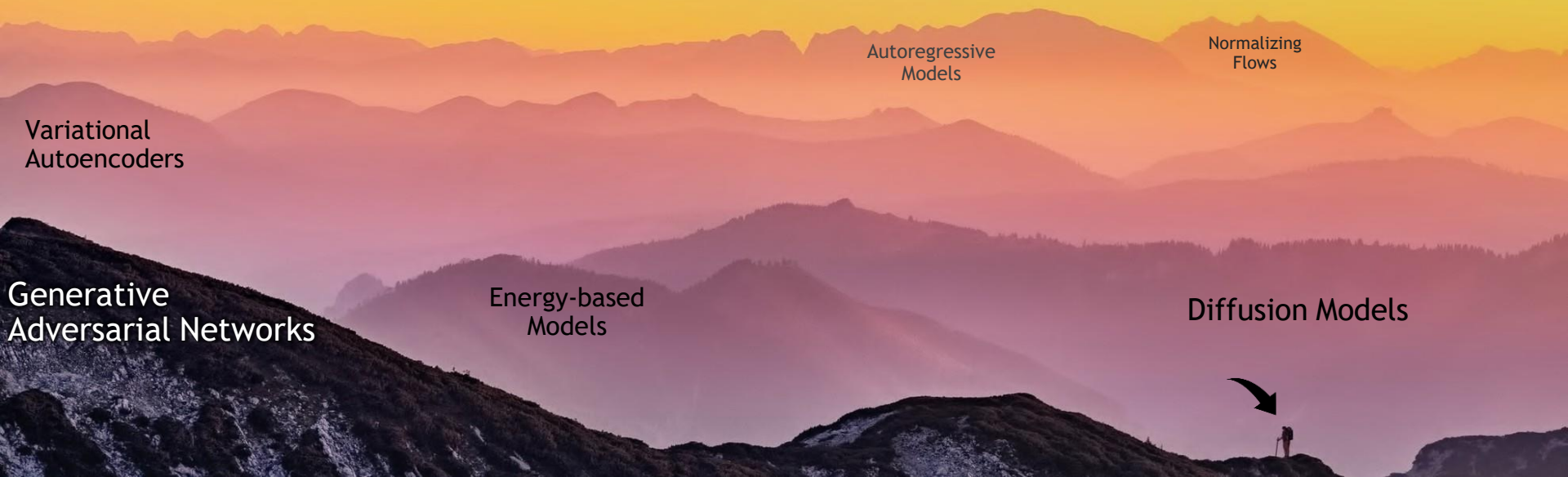
Normalizing
Flows

Generative
Adversarial Networks

Energy-based
Models

Diffusion Models

The Landscape of Deep Generative Models



Variational
Autoencoders

Autoregressive
Models

Normalizing
Flows

Generative
Adversarial Networks

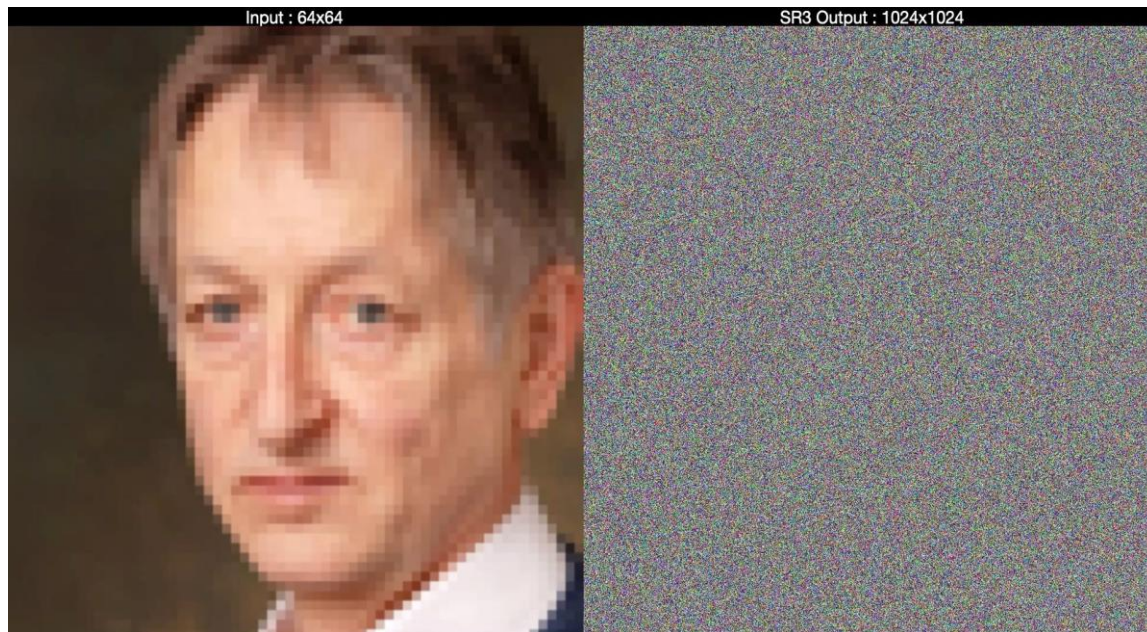
Energy-based
Models

Diffusion Models

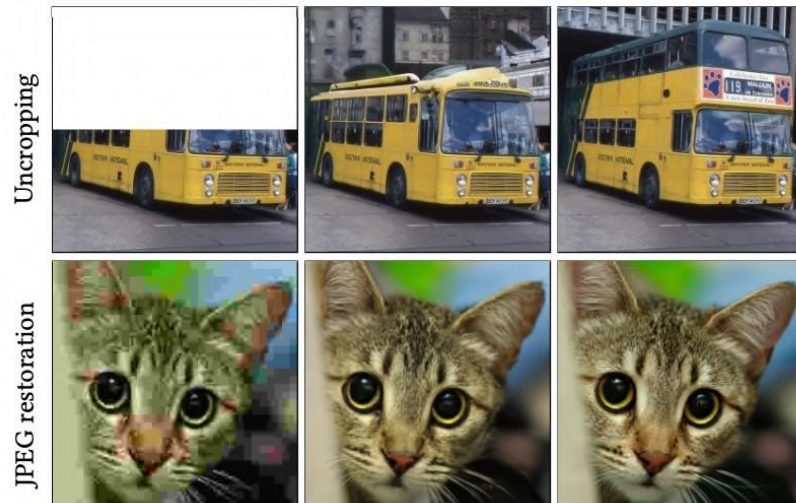
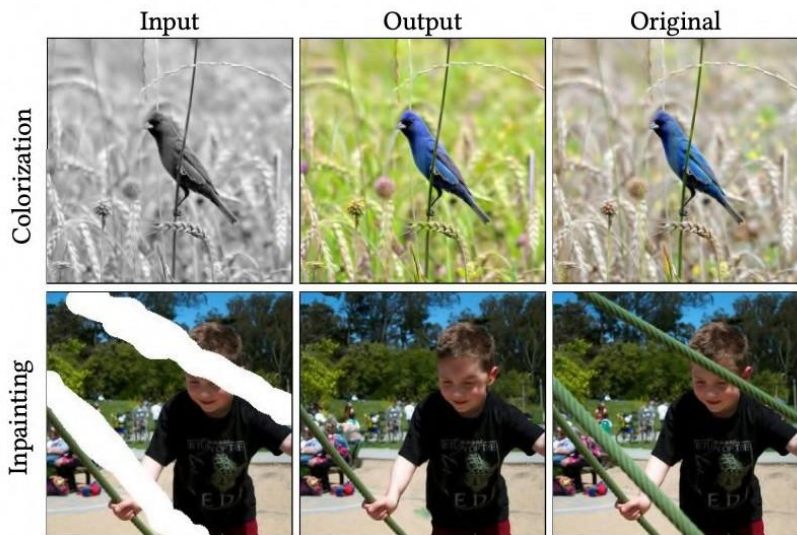
AI Art



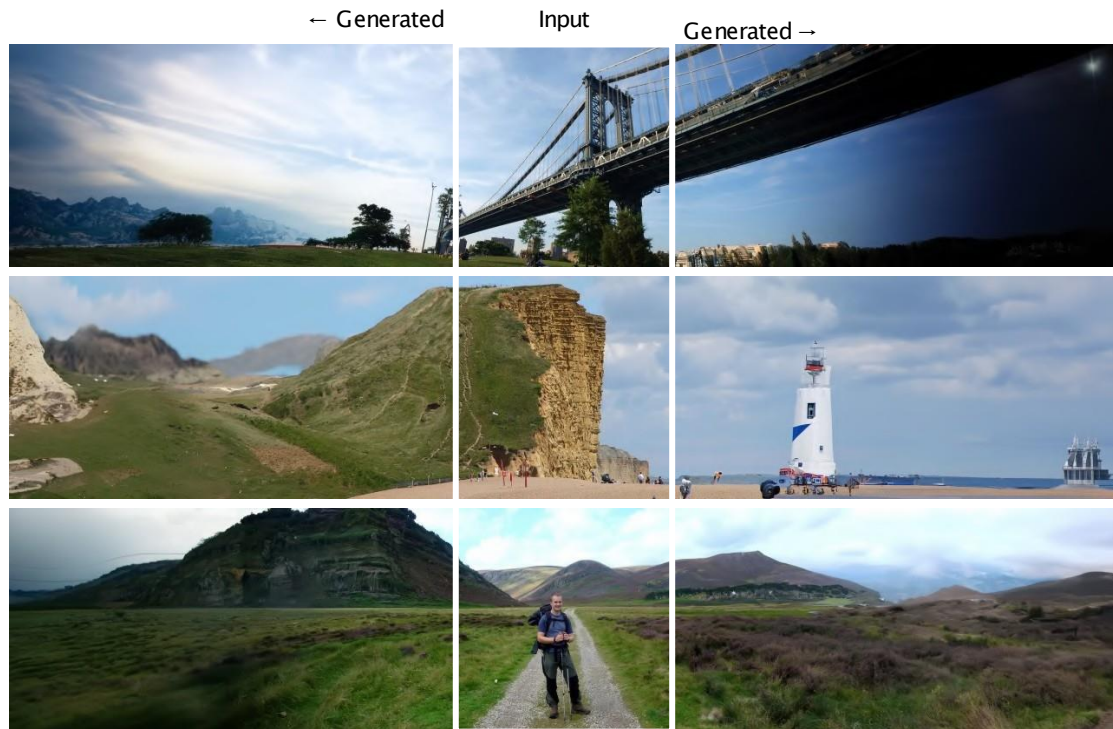
Applications: Super-resolution



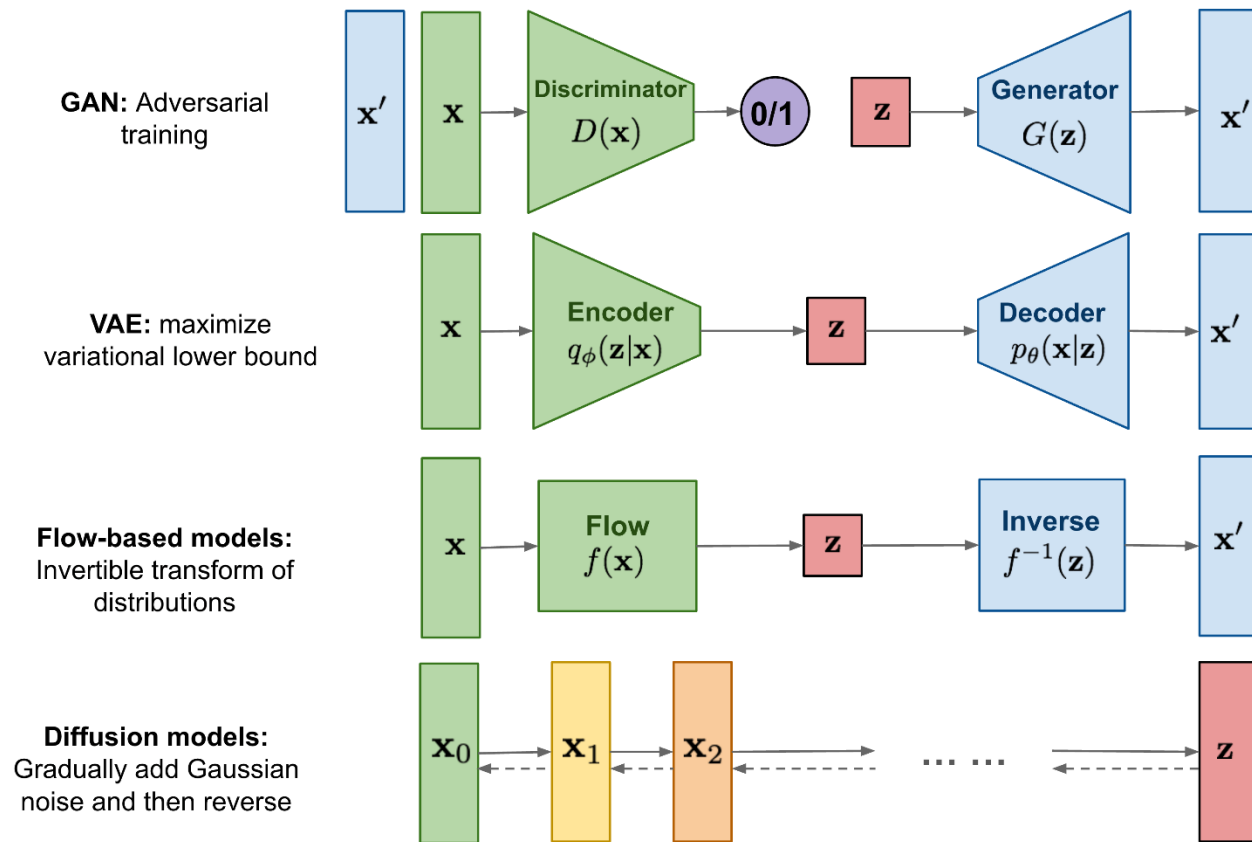
Applications: Colorization, Inpainting, Restoration



Applications: Outfilling

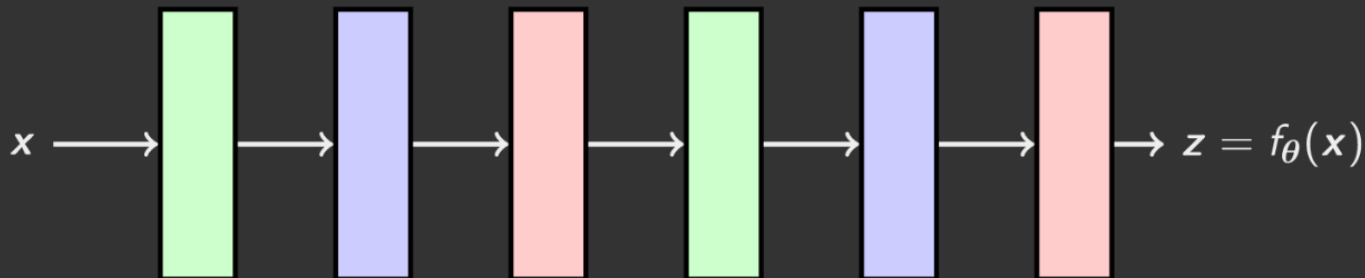


Overview of Generative Models



Normalizing Flows

FLOWS: MAIN IDEA



Generally: $\mathbf{z} \sim p_Z(\mathbf{z})$

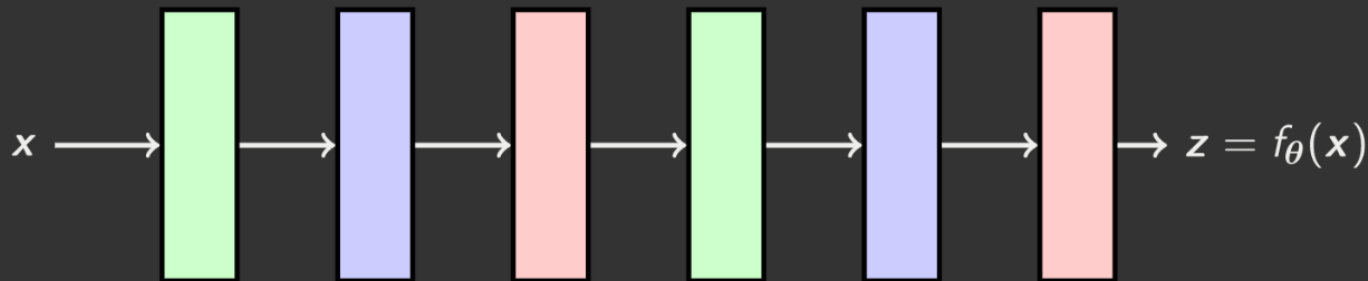
Normalizing Flow: $\mathbf{z} \sim \mathcal{N}(0, 1)$

Key questions:

- How to train?
- How to evaluate $p_{\theta}(x)$?
- How to sample?



FLOWS: TRAINING



$$\max_{\theta} \sum_i \log p_{\theta} \left(x^{(i)} \right)$$

Need $f(\cdot)$ to be invertible and differentiable

CHANGE OF VARIABLES FORMULA

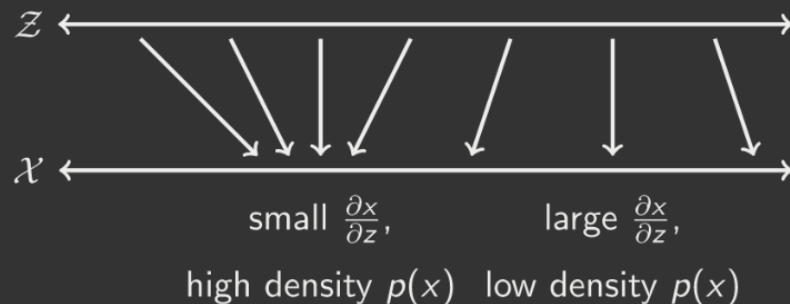
Let f^{-1} denote a differentiable, bijective mapping from space $\mathcal{Z}$ to space $\mathcal{X}$, i.e., it must be 1-to-1 and cover all of $\mathcal{X}$.

Since f^{-1} defines a one-to-one correspondence between values $\mathbf{z} \in \mathcal{Z}$ and $\mathbf{x} \in \mathcal{X}$, we can think of it as a change-of-variables transformation.

Change-of-Variables Formula from probability theory. If $\mathbf{z} = f(\mathbf{x})$, then

$$p_{\mathcal{X}}(\mathbf{x}) = p_{\mathcal{Z}}(\mathbf{z}) \left| \det \left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right|$$

Intuition for the Jacobian term:



FLOWS: TRAINING

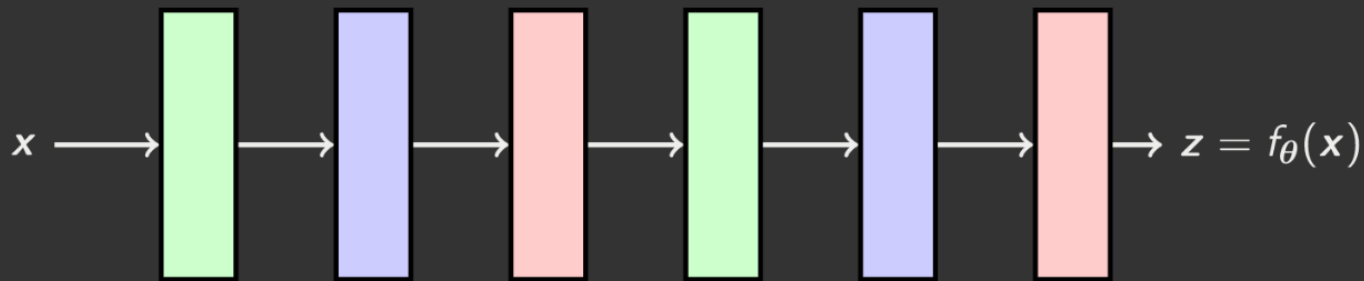
$$\max_{\theta} \sum_i \log p_{\theta}(x^{(i)})$$

$$\begin{aligned} \mathbf{z} &= f_{\theta}(\mathbf{x}) \\ p_{\theta}(\mathbf{x}) &= p_Z(\mathbf{z}) \left| \det \left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}} \right) \right| \\ &= p_Z(f_{\theta}(\mathbf{x})) \left| \det \left(\frac{\partial f_{\theta}(\mathbf{x})}{\partial \mathbf{x}} \right) \right| \end{aligned}$$

$$\max_{\theta} \sum_i \log p_{\theta}(x^{(i)}) = \max_{\theta} \sum_i \log p_Z(f_{\theta}(\mathbf{x}^{(i)})) + \log \left| \det \left(\frac{\partial f_{\theta}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

Assuming we have an expression for p_Z , we can optimize this with Stochastic Gradient Descent

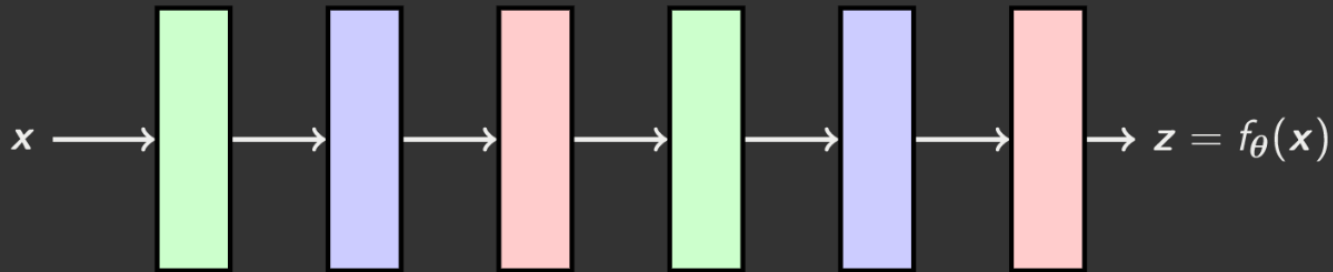
FLOWS: SAMPLING



Step 1: sample $\mathbf{z} \sim p_Z(\mathbf{z})$

Step 2: $\mathbf{x} = f_{\theta}^{-1}(\mathbf{z})$

CHANGE OF VARIABLES FORMULA



Problems?

- The mapping f needs to be invertible, with an easy-to-compute inverse.
- It needs to be differentiable, so that the Jacobian $\frac{\partial x}{\partial z}$ is defined.
- We need to be able to compute the (log) determinant.

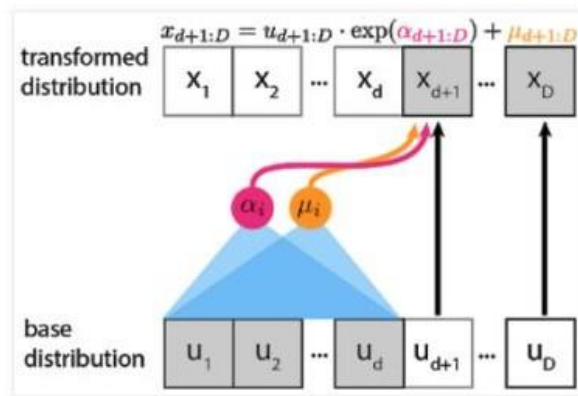
Real-NVP

Real-Non Volume Preserving Flows

Let $1 < k < d$, $\odot$ element-wise multiplication and t, s two mappings $\mathbb{R}^k \rightarrow \mathbb{R}^{d-k}$. R-NVPs are defined as

$$\mathbf{y}_{1:k} = \mathbf{z}_{1:k},$$

$$\mathbf{y}_{k+1:d} = \mathbf{z}_{k+1:d} \odot \exp(s(\mathbf{z}_{1:k})) + t(\mathbf{z}_{1:k})$$



Schematic of the data split. $\mathbf{u}$ is $\mathbf{z}$, $\mathbf{x}$ is $\mathbf{y}$, α is s and μ is t in the equations.

Real-Non Volume Preserving Flows

Forward transformation (sampling) is very straight-forward, copy first part of dimensions and scaling and shifting the other part by learnable parameters. This is fully parallelizable, and inverse transformation (inference) is like so as well!

$$\mathbf{z}_{1:k} = \mathbf{y}_{1:k}$$

$$\mathbf{z}_{k+1:d} = (\mathbf{y}_{k+1:d} - t(\mathbf{y}_{1:k})) / \exp(s(\mathbf{y}_{1:k}))$$

This operation is the **affine coupling layer**.

Real-Non Volume Preserving Flows

The determinant of this layer is as simple as:

$$\frac{\partial y}{\partial x^T} = \begin{bmatrix} \mathbb{I}_d & 0 \\ \frac{\partial y_{d+1:D}}{\partial x_{1:d}^T} & \text{diag}(\exp[s(x_{1:d})]) \end{bmatrix}$$

Where $s(x_{1:d})$ is the predicted scale vector, easy peasy. This means it is not necessary to compute s or t Jacobians, so s and t can be arbitrarily complex (e.g. Resnets).

Real-Non Volume Preserving Flows

The determinant of this layer is as simple as:

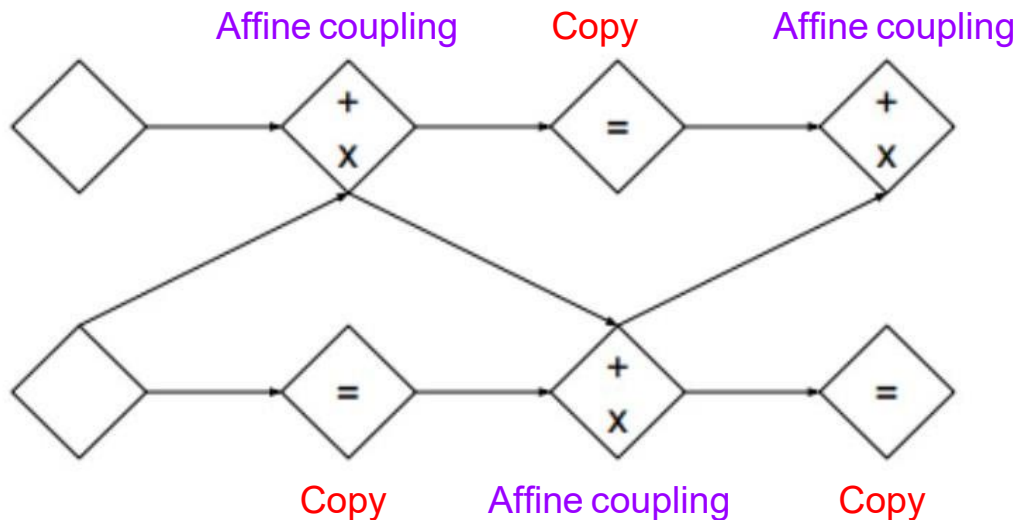
Copy

$$\frac{\partial y}{\partial x^T} = \begin{bmatrix} \boxed{\mathbb{I}_d} & 0 \\ \frac{\partial y_{d+1:D}}{\partial x_{1:d}^T} & \boxed{\text{diag}(\exp[s(x_{1:d})])} \end{bmatrix} \text{Affine coupling}$$

Where $s(x_{1:d})$ is the predicted scale vector, easy peasy. This means it is not necessary to compute s or t Jacobians, so s and t can be arbitrarily complex (e.g. Resnets).

Real-NVP Permutations

Due to certain dimensions being just copied and forwarded, every intermediate flow permutes the state of the intermediate vector. This way all dimensions are effectively transformed after enough levels.



GLOW

GLOW

The structure follows the same factorizing multi-scale structure as in Real-NVP, but proposing their own step of flow. Notation as in the paper regarding what we are going to see:

$$\mathbf{z} \sim p_{\theta}(\mathbf{z})$$

$$\mathbf{x} \sim g_{\theta}(\mathbf{z})$$

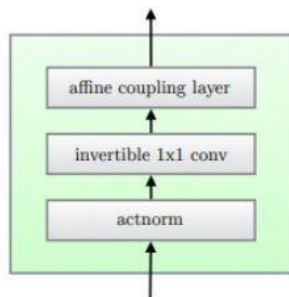
$$\mathbf{x} \xleftrightarrow{\mathbf{f}_1} \mathbf{h}_1 \xleftrightarrow{\mathbf{f}_2} \mathbf{h}_2 \cdots \xleftrightarrow{\mathbf{f}_K} \mathbf{z}$$

The objective (loss), as it is an NF-based model, is the negative log-likelihood (that simple!)

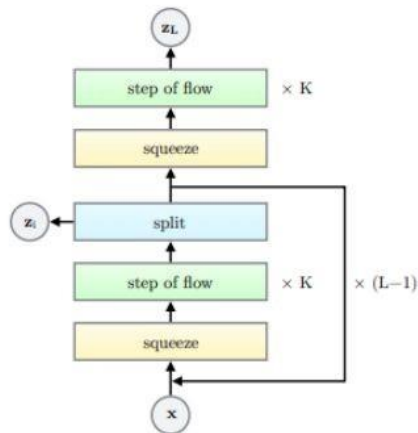
$$L(D) = \frac{1}{N} \sum_{i=1}^N -\log p_{\theta}(\mathbf{x}_i)$$

GLOW

The structure follows the same factorizing multi-scale structure as in Real-NVP, but proposing their own step of flow. There a total of K flows, with a total of L levels.



(a) One step of our flow.

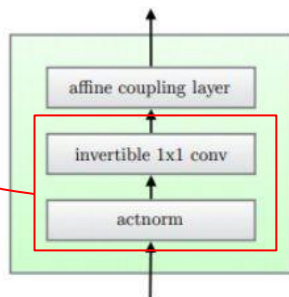


(b) Multi-scale architecture (Dinh et al., 2016).

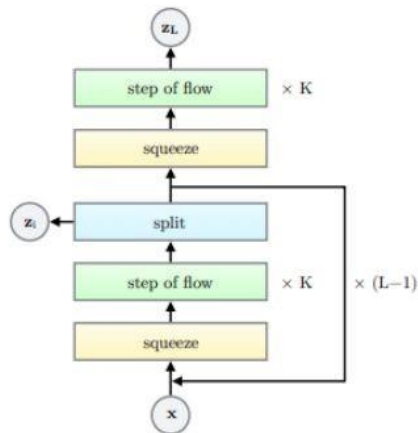
GLOW

The structure follows the same factorizing multi-scale structure as in Real-NVP, but proposing their own step of flow. There a total of K flows, with a total of L levels.

Improvements over Real-NVP.



(a) One step of our flow.



(b) Multi-scale architecture (Dinh et al., 2016).

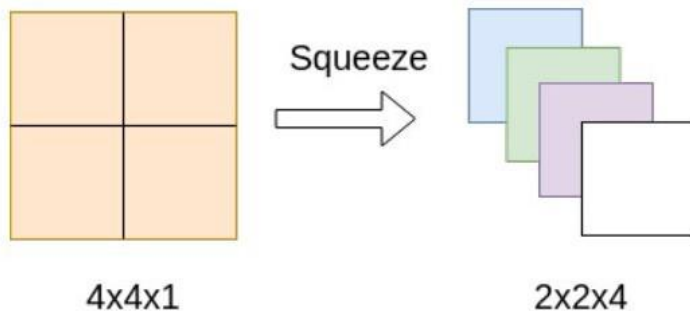
GLOW step of flow formulations

Table 1: The three main components of our proposed flow, their reverses, and their log-determinants. Here, $\mathbf{x}$ signifies the input of the layer, and $\mathbf{y}$ signifies its output. Both $\mathbf{x}$ and $\mathbf{y}$ are tensors of shape $[h \times w \times c]$ with spatial dimensions (h, w) and channel dimension c . With (i, j) we denote spatial indices into tensors $\mathbf{x}$ and $\mathbf{y}$. The function $\text{NN}()$ is a nonlinear mapping, such as a (shallow) convolutional neural network like in ResNets (He et al., 2016) and RealNVP (Dinh et al., 2016).

Description	Function	Reverse Function	Log-determinant
Actnorm. See Section 3.1.	$\forall i, j : \mathbf{y}_{i,j} = \mathbf{s} \odot \mathbf{x}_{i,j} + \mathbf{b}$	$\forall i, j : \mathbf{x}_{i,j} = (\mathbf{y}_{i,j} - \mathbf{b})/\mathbf{s}$	$h \cdot w \cdot \text{sum}(\log \mathbf{s})$
Invertible 1×1 convolution. $\mathbf{W} : [c \times c]$. See Section 3.2.	$\forall i, j : \mathbf{y}_{i,j} = \mathbf{W}\mathbf{x}_{i,j}$	$\forall i, j : \mathbf{x}_{i,j} = \mathbf{W}^{-1}\mathbf{y}_{i,j}$	$h \cdot w \cdot \log \det(\mathbf{W}) $ or $h \cdot w \cdot \text{sum}(\log \mathbf{s})$ (see eq. (10))
Affine coupling layer. See Section 3.3 and (Dinh et al., 2014)	$\mathbf{x}_a, \mathbf{x}_b = \text{split}(\mathbf{x})$ $(\log \mathbf{s}, \mathbf{t}) = \text{NN}(\mathbf{x}_b)$ $\mathbf{s} = \exp(\log \mathbf{s})$ $\mathbf{y}_a = \mathbf{s} \odot \mathbf{x}_a + \mathbf{t}$ $\mathbf{y}_b = \mathbf{x}_b$ $\mathbf{y} = \text{concat}(\mathbf{y}_a, \mathbf{y}_b)$	$\mathbf{y}_a, \mathbf{y}_b = \text{split}(\mathbf{y})$ $(\log \mathbf{s}, \mathbf{t}) = \text{NN}(\mathbf{y}_b)$ $\mathbf{s} = \exp(\log \mathbf{s})$ $\mathbf{x}_a = (\mathbf{y}_a - \mathbf{t})/\mathbf{s}$ $\mathbf{x}_b = \mathbf{y}_b$ $\mathbf{x} = \text{concat}(\mathbf{x}_a, \mathbf{x}_b)$	$\text{sum}(\log(\mathbf{s}))$

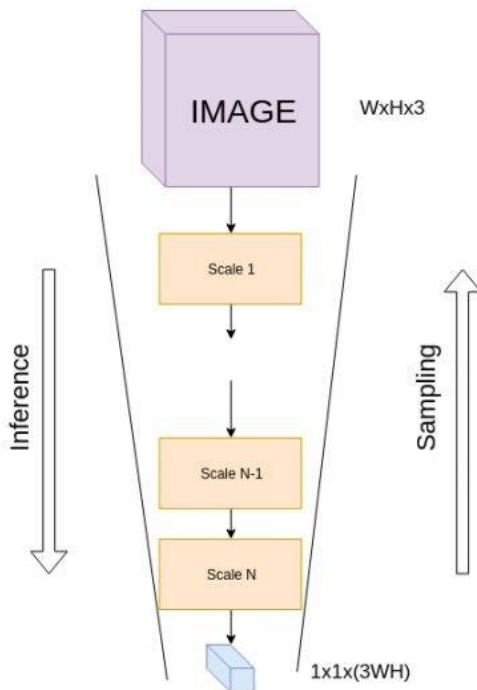
GLOW Multi-scale structure

Squeeze from image dimensions to prior dimensions ("vectorize the tensor") with a simple **reshape**. Each level of squeezing is a level of the multi-scale scheme. Each level contains K flows.



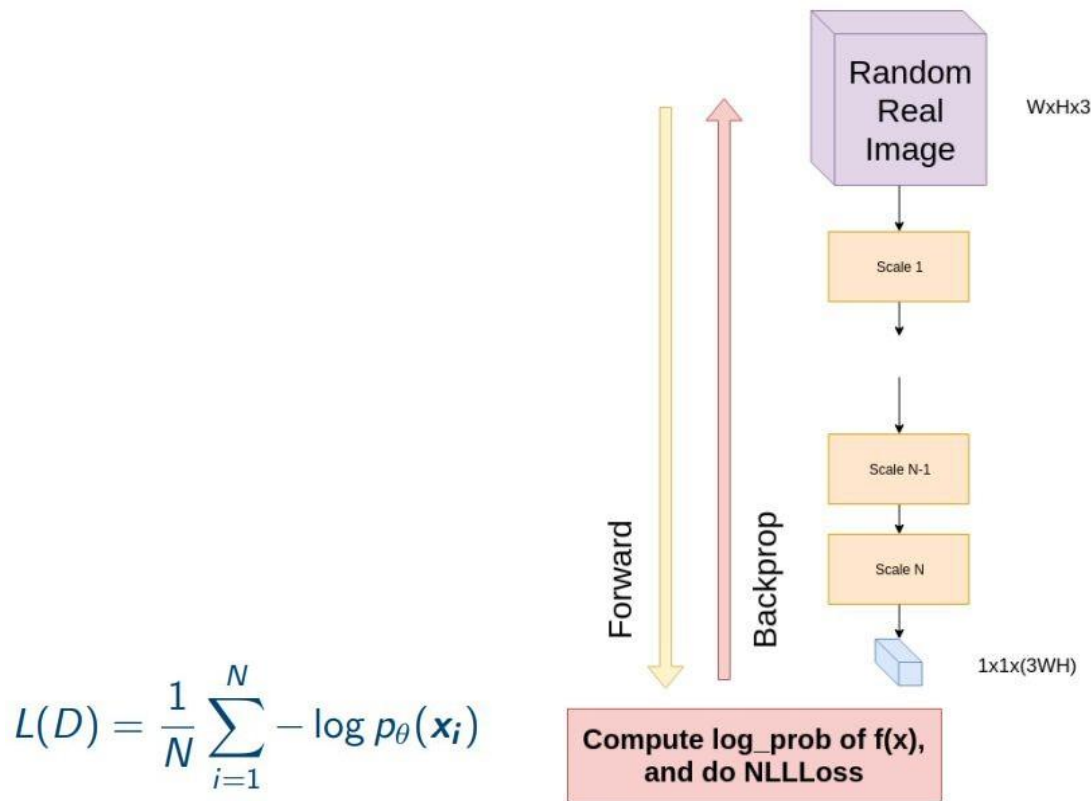
GLOW Multi-scale structure

This is a very powerful structure: we can infer z features from an image and we can sample an image from z features, EITHER WAY!



GLOW Multi-scale structure

Additionally, during training loss is computed in the simplistic Z space!



GLOW Results on CelebA

We can sample with a temperature factor T that simply multiplies the scale s in the coupling layers.

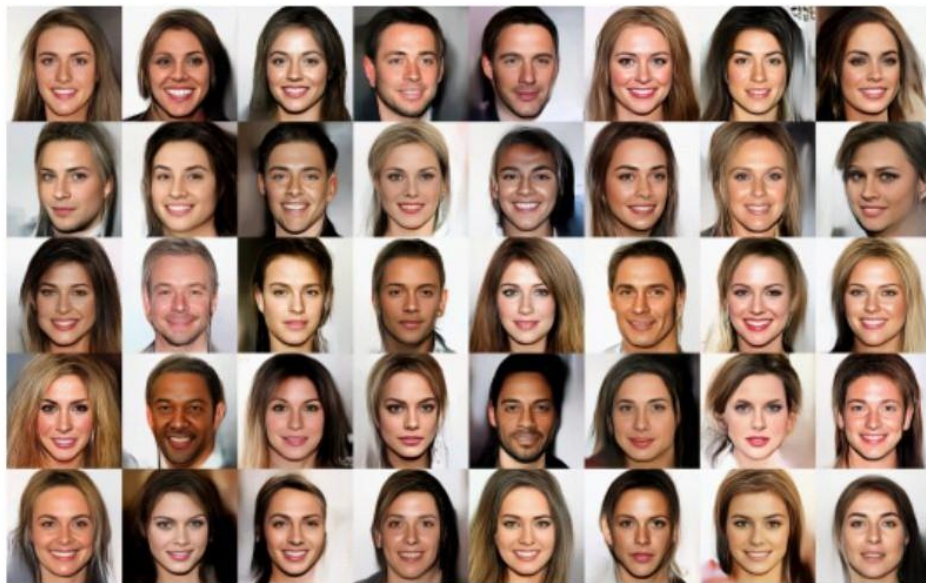


Figure 4: Random samples from the model, with temperature 0.7

GLOW Results with temperature

We can sample with a temperature factor T that simply multiplies the scale s in the coupling layers. Sample quality and diversity varies with T factor, being 0.7 a sweet spot for the authors in this work.



Figure 8: Effect of change of temperature. From left to right, samples obtained at temperatures 0, 0.25, 0.6, 0.7, 0.8, 0.9, 1.0

GLOW Results interpolating Z

We can infer two real images onto their respective latent representations and sample from their linear interpolations to smoothly mix both images.



Figure 5: Linear interpolation in latent space between real images

GLOW Semantic Manipulation

In celebA there is a binary label per attribute (absence/presence): blonde, smiling, etc. So there are 30.000 binary labels per attribute (from training data).

The z_{pos} average latent vector is calculated for images with the attribute and z_{neg} for images without it. The difference $z_{pos} - z_{neg}$ can be used as a direction for manipulating a certain attribute.

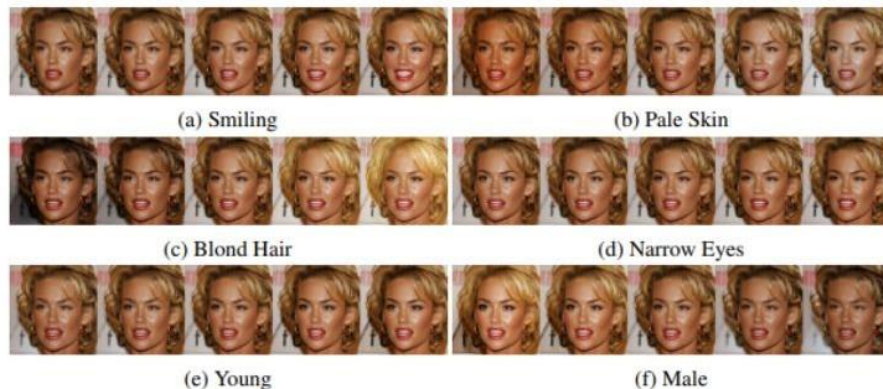


Figure 6: Manipulation of attributes of a face. Each row is made by interpolating the latent code of an image along a vector corresponding to the attribute, with the middle image being the original image

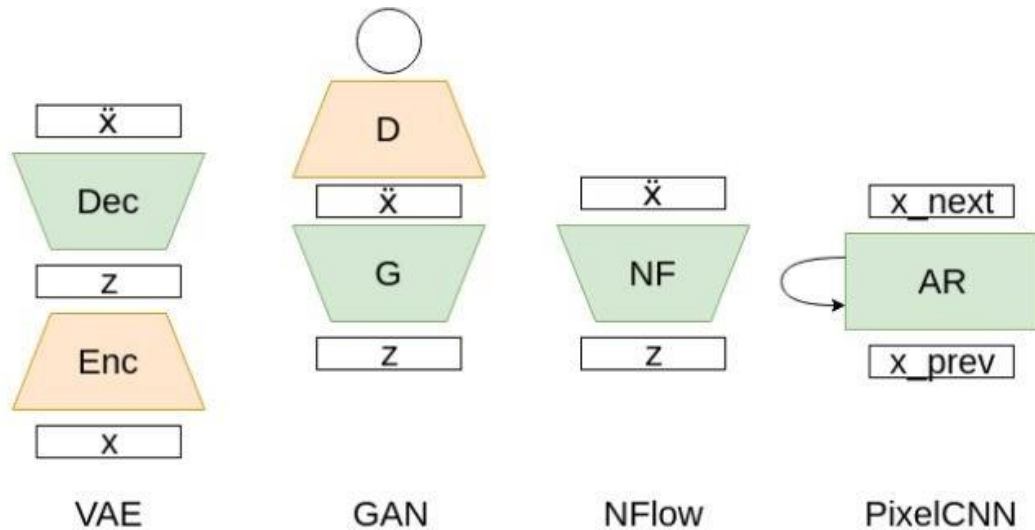
GLOW's depth dependency

Without enough depth, there is a lack of capacity to generate long-range dependencies.

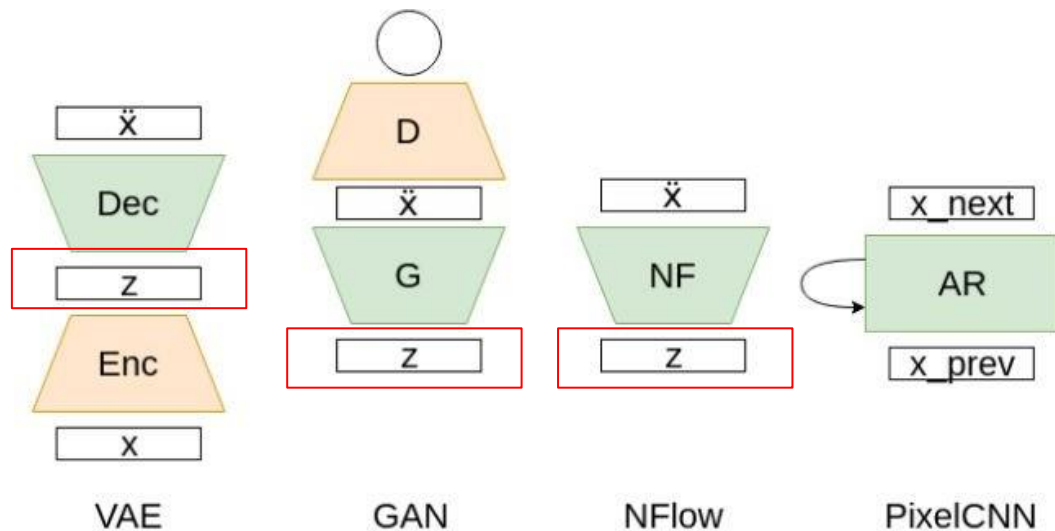


Figure 9: Samples from shallow model on left vs deep model on right. Shallow model has $L = 4$ levels, while deep model has $L = 6$ levels

Models comparison

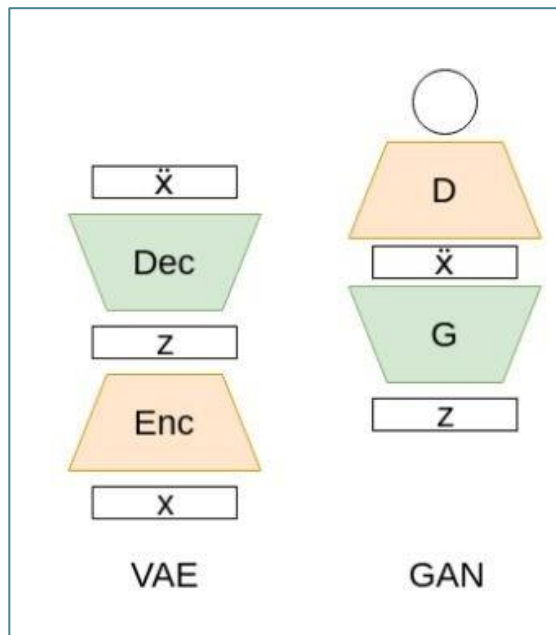


Models comparison

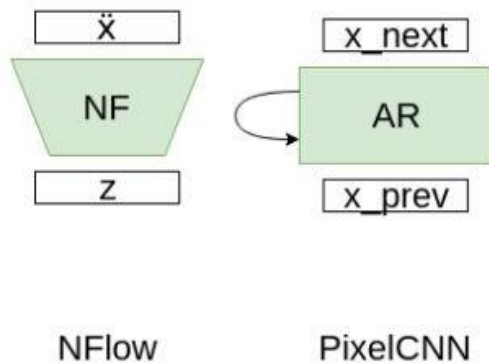


Latent space $\rightarrow$ feature disentangle, we can navigate and control better what we generate

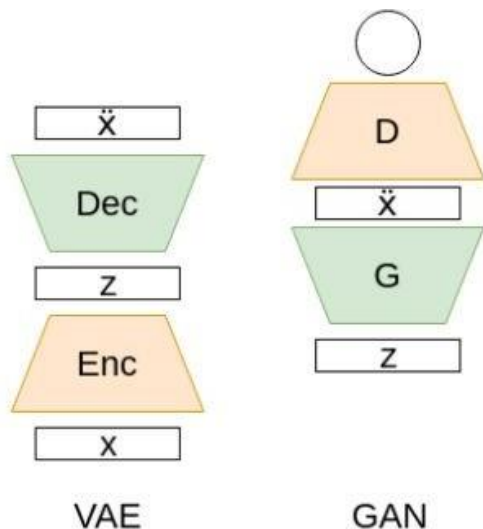
Models comparison



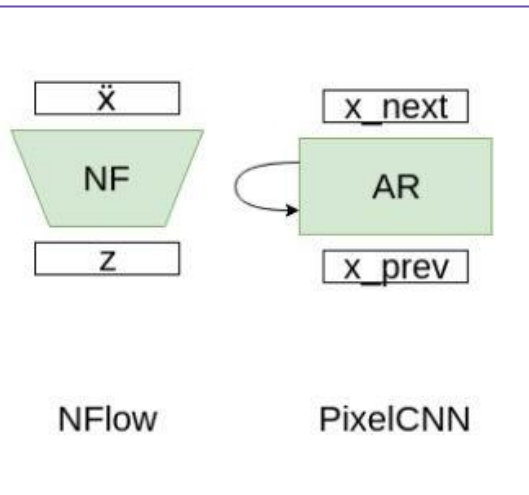
No direct likelihood measurement, just an approximation or qualitative insights.



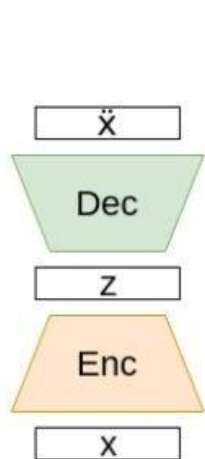
Models comparison



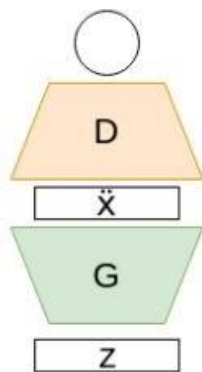
Direct likelihood values, direct objective “good fit” measurements.



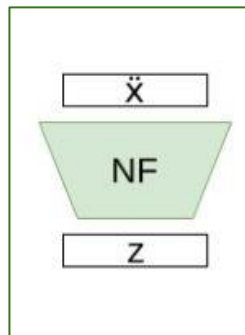
Models comparison



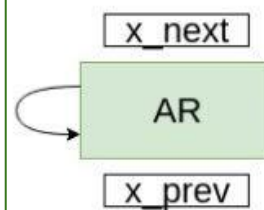
VAE



GAN



NFlow

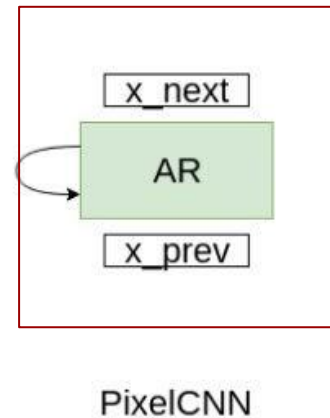
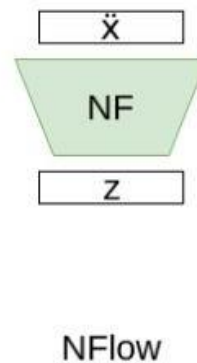
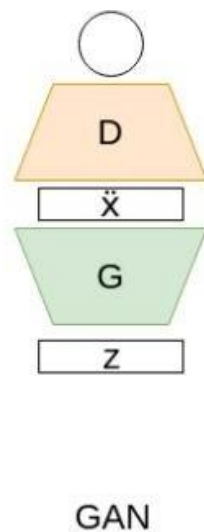
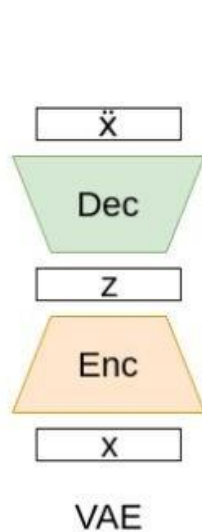


PixelCNN

Very deep models (many flow steps), need much much memory and compute power to get best results.



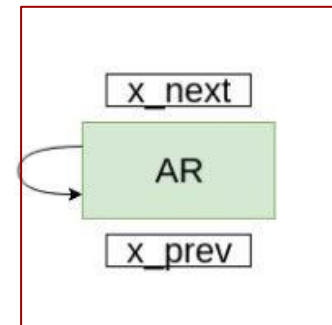
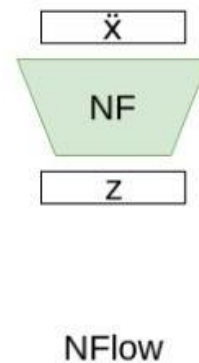
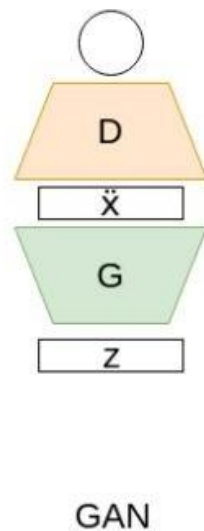
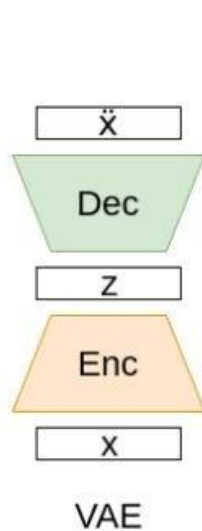
Models comparison



Fast training.



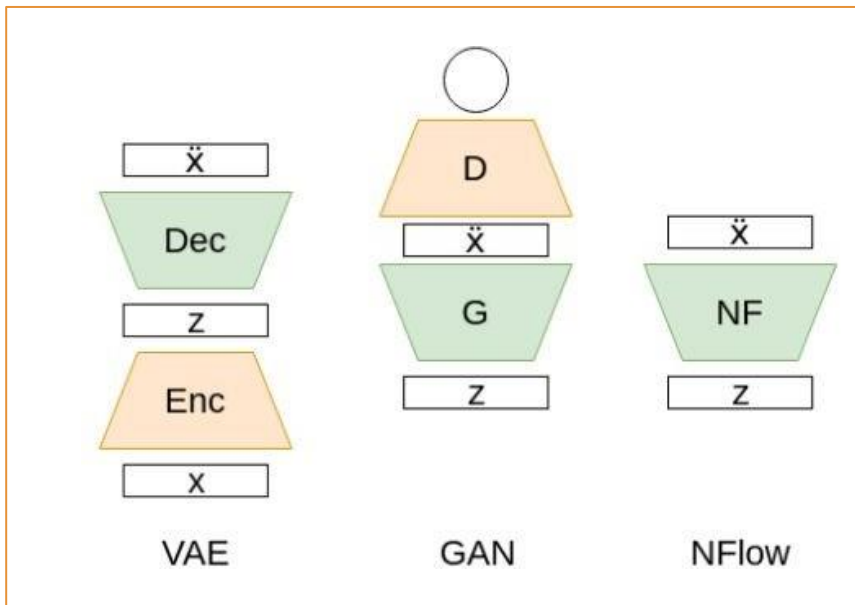
Models comparison



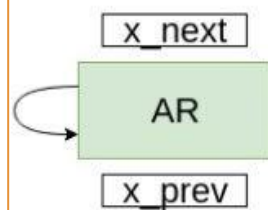
Slow auto-regressive sampling.



Models comparison



One-shot predictions,
quickest to sample from.



Denoising Diffusion Probabilistic Models

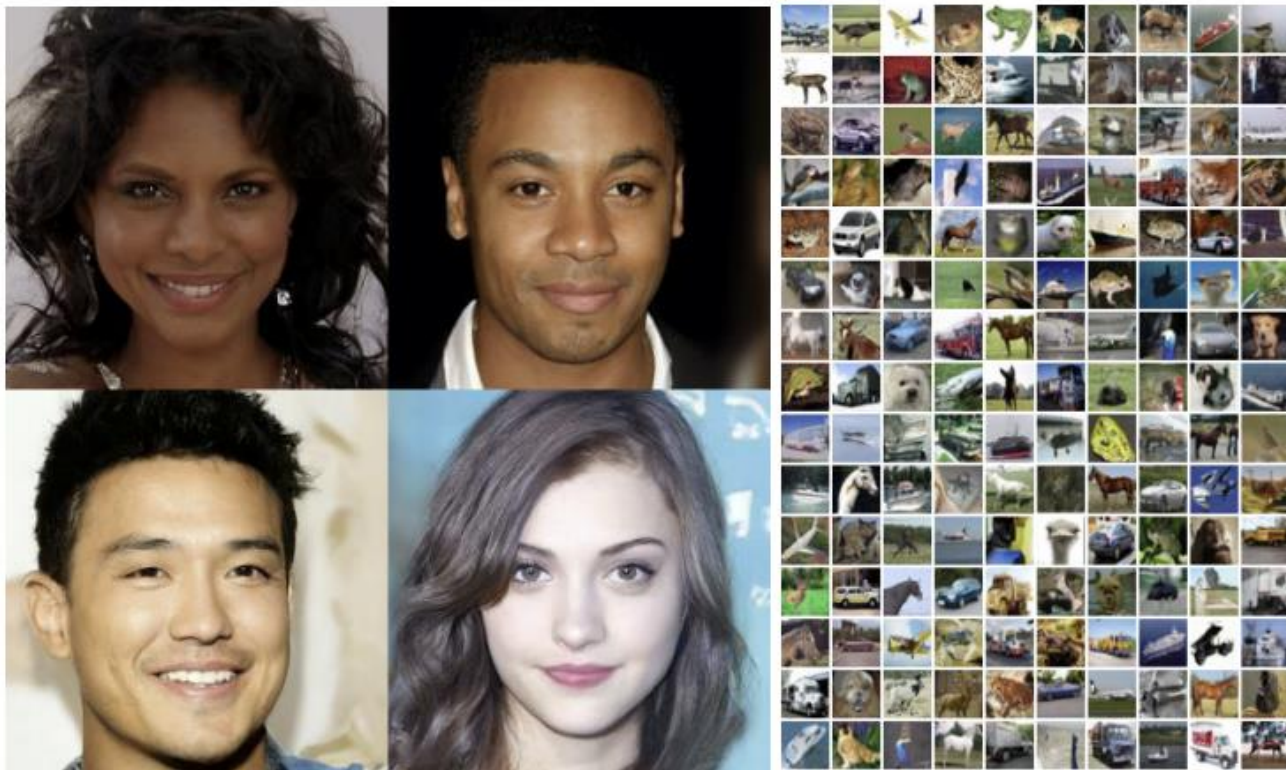


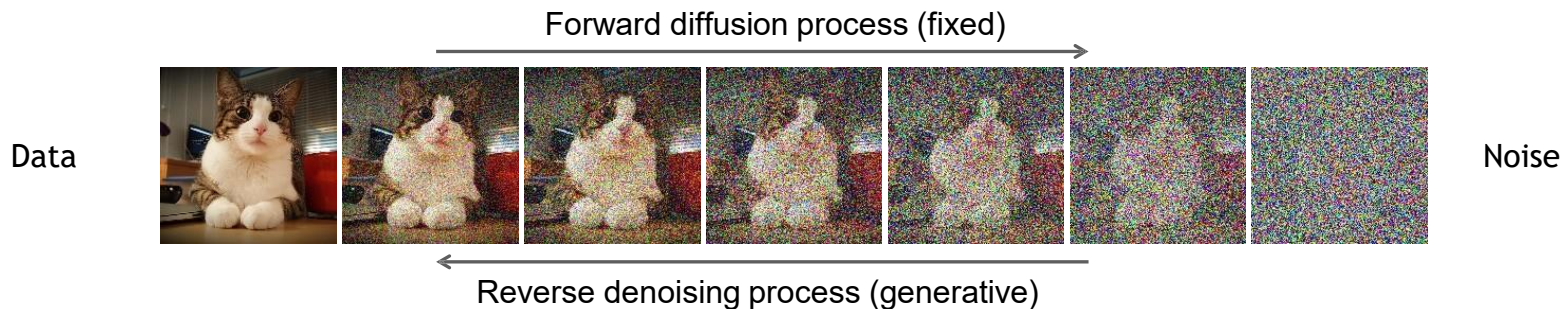
Figure 1: Generated samples on CelebA-HQ 256×256 (left) and unconditional CIFAR10 (right)

Denoising Diffusion Models

Learning to generate by denoising

Denoising diffusion models consist of two processes:

- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising



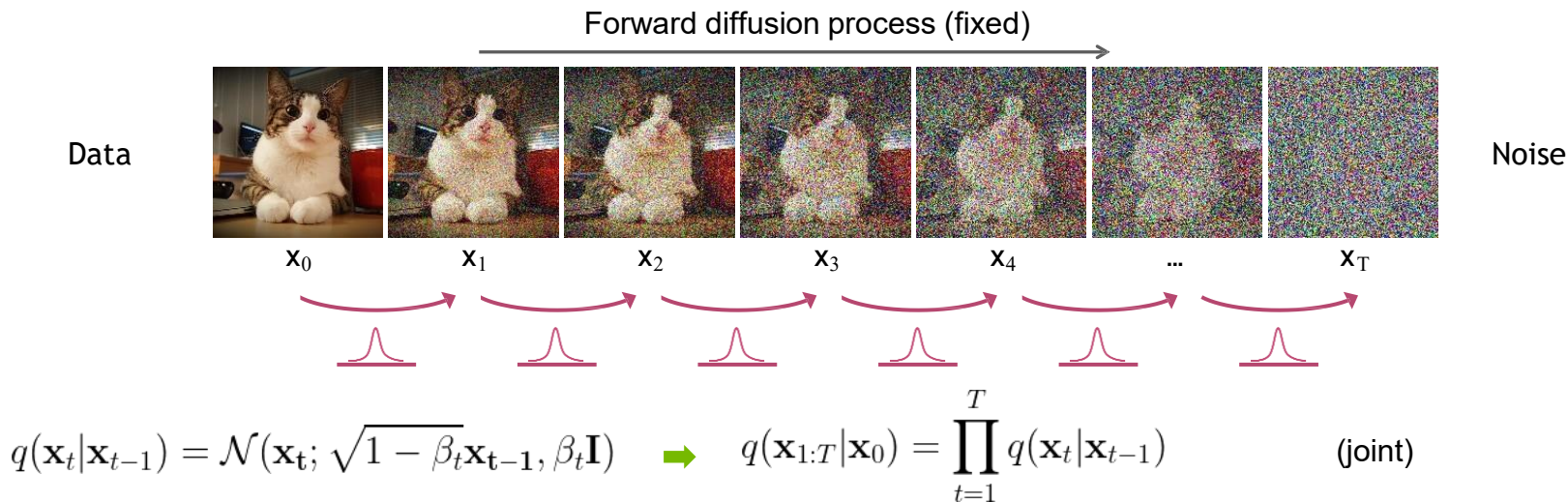
[Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015](#)

[Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020](#)

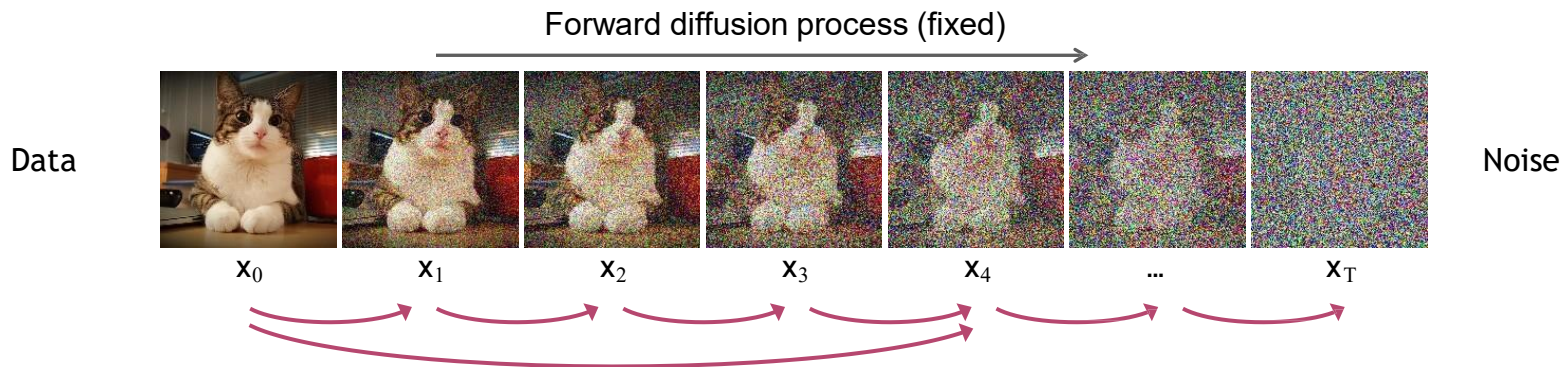
[Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021](#)

Forward Diffusion Process

The formal definition of the forward process in T steps:



Diffusion Kernel



Define $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ ➡ $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$ (Diffusion Kernel)

For sampling: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

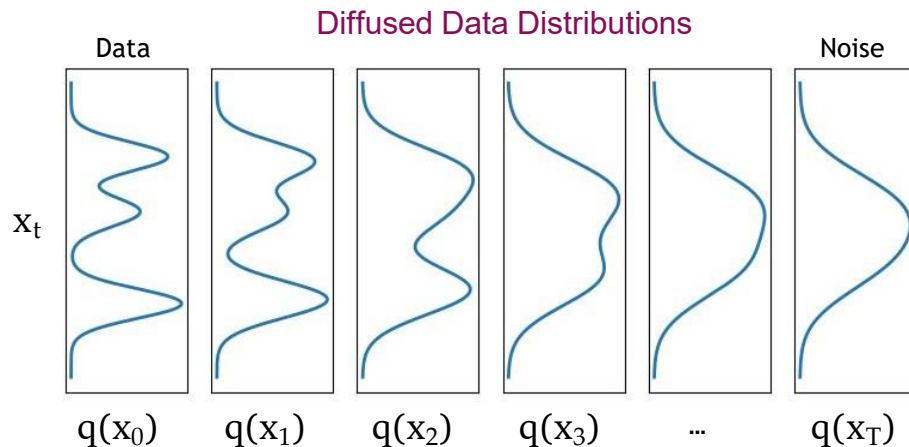
β_t values schedule (i.e., the noise schedule) is designed such that $\bar{\alpha}_T \rightarrow 0$ and $q(\mathbf{x}_T | \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

What happens to a distribution in the forward diffusion?

So far, we discussed the diffusion kernel $q(\mathbf{x}_t|\mathbf{x}_0)$ but what about $q(\mathbf{x}_t)$?

$$\underbrace{q(\mathbf{x}_t)}_{\text{Diffused data dist.}} = \int \underbrace{q(\mathbf{x}_0, \mathbf{x}_t)}_{\text{Joint dist.}} d\mathbf{x}_0 = \int \underbrace{q(\mathbf{x}_0)}_{\text{Input data dist.}} \underbrace{q(\mathbf{x}_t|\mathbf{x}_0)}_{\text{Diffusion kernel}} d\mathbf{x}_0$$

The diffusion kernel is Gaussian convolution.



We can sample $\mathbf{x}_t \sim q(\mathbf{x}_t)$ by first sampling $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ and then sampling $\mathbf{x}_t \sim q(\mathbf{x}_t|\mathbf{x}_0)$ (i.e., ancestral sampling).

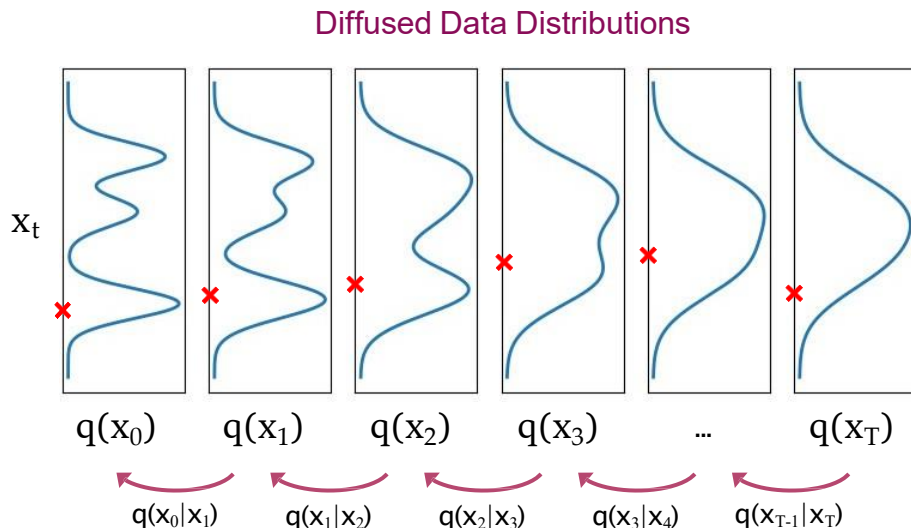
Generative Learning by Denoising

Recall, that the diffusion parameters are designed such that $q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Generation:

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Iteratively sample $\mathbf{x}_{t-1} \sim \underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t)}_{\text{True Denoising Dist.}}$

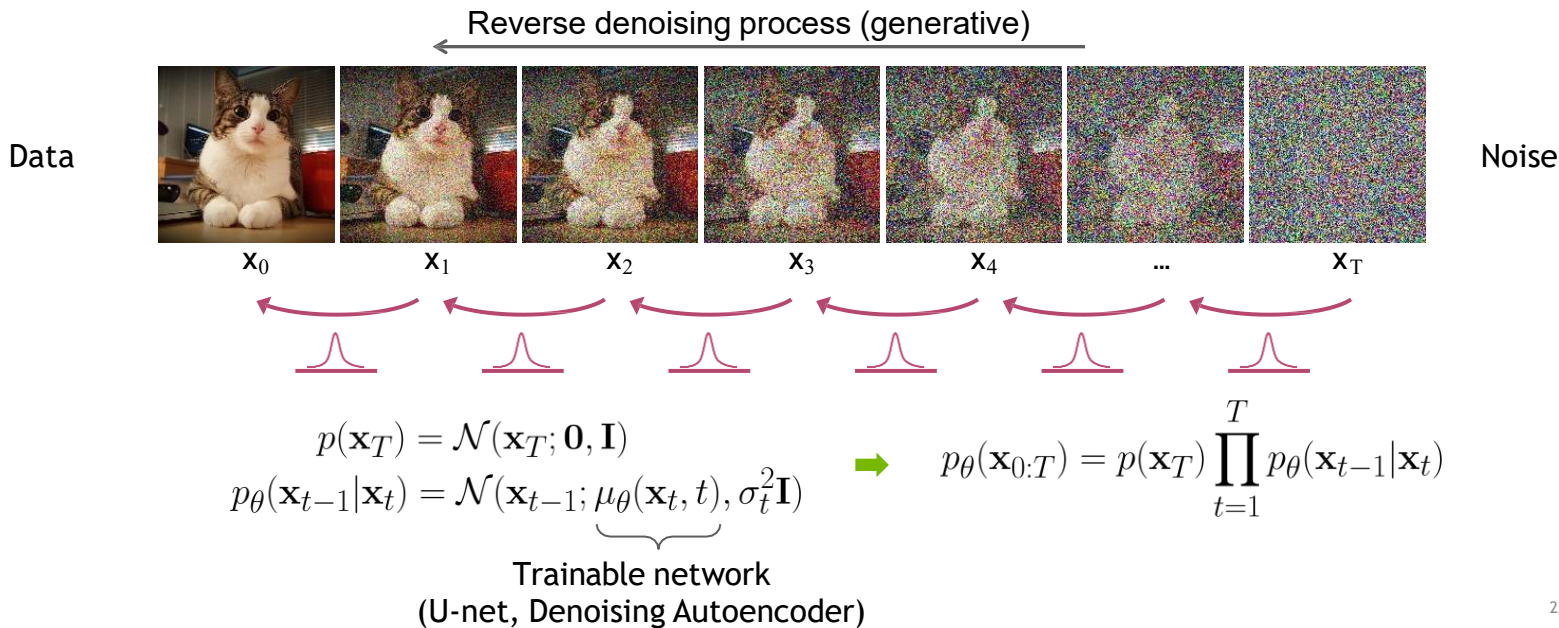


In general, $q(\mathbf{x}_{t-1}|\mathbf{x}_t) \propto q(\mathbf{x}_{t-1})q(\mathbf{x}_t|\mathbf{x}_{t-1})$ is intractable.

Can we approximate $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$? Yes, we can use a **Normal distribution** if β_t is small in each forward diffusion step.

Reverse Denoising Process

Formal definition of forward and reverse processes in T steps:



Learning Denoising Model

Variational upper bound

For training, we can form variational upper bound that is commonly used for training variational autoencoders:

$$\mathbb{E}_{q(\mathbf{x}_0)} [-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L$$

[Sohl-Dickstein et al. ICML 2015](#) and [Ho et al. NeurIPS 2020](#) show that:

$$L = \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

where $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is the tractable posterior distribution:

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}),$$

$$\text{where } \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{1 - \beta_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t \text{ and } \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

Parameterizing the Denoising Model

Since both $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ and $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ are Normal distributions, the KL divergence has a simple form:

$$L_{t-1} = D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) || p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} ||\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)||^2 \right] + C$$

Recall that $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$. [Ho et al. NeurIPS 2020](#) observe that:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right)$$

They propose to represent the mean of the denoising model using a *noise-prediction* network:

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

With this parameterization

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\frac{\beta_t^2}{2\sigma_t^2(1 - \beta_t)(1 - \bar{\alpha}_t)} ||\epsilon - \underbrace{\epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)}_{\mathbf{x}_t}}||^2 \right] + C$$

Training Objective Weighting

Trading likelihood for perceptual quality

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\underbrace{\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}}_{\lambda_t} \|\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t} \epsilon, t)}_{\mathbf{x}_t}\|^2 \right]$$

The time dependent λ_t ensures that the training objective is weighted properly for the maximum data likelihood training.

However, this weight is often very large for small t's.

[Ho et al. NeurIPS 2020](#) observe that simply setting $\lambda_t = 1$ improves sample quality. So, they propose to use:

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1, T)} \left[\|\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t} \epsilon}_{\mathbf{x}_t}, t)\|^2 \right]$$

For more advanced weighting see [Choi et al., Perception Prioritized Training of Diffusion Models, CVPR 2022](#).

Summary

Training and Sample Generation

Algorithm 1 Training

- 1: **repeat**
 - 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
 - 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 5: Take gradient descent step on
 $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$
 - 6: **until** converged
-

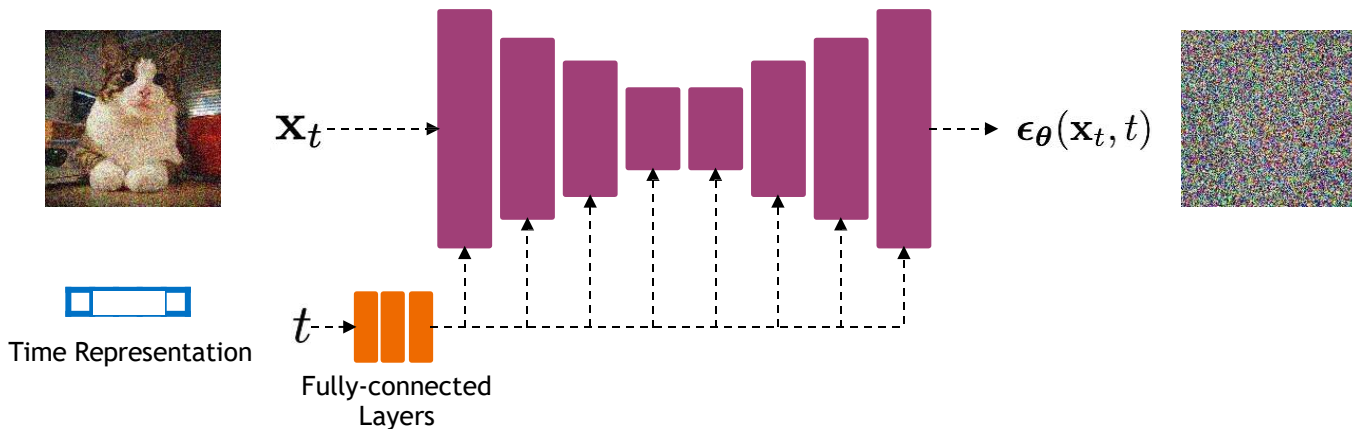
Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 2: **for** $t = T, \dots, 1$ **do**
 - 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
 - 5: **end for**
 - 6: **return** $\mathbf{x}_0$
-

Implementation Considerations

Network Architectures

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_{\theta}(\mathbf{x}_t, t)$

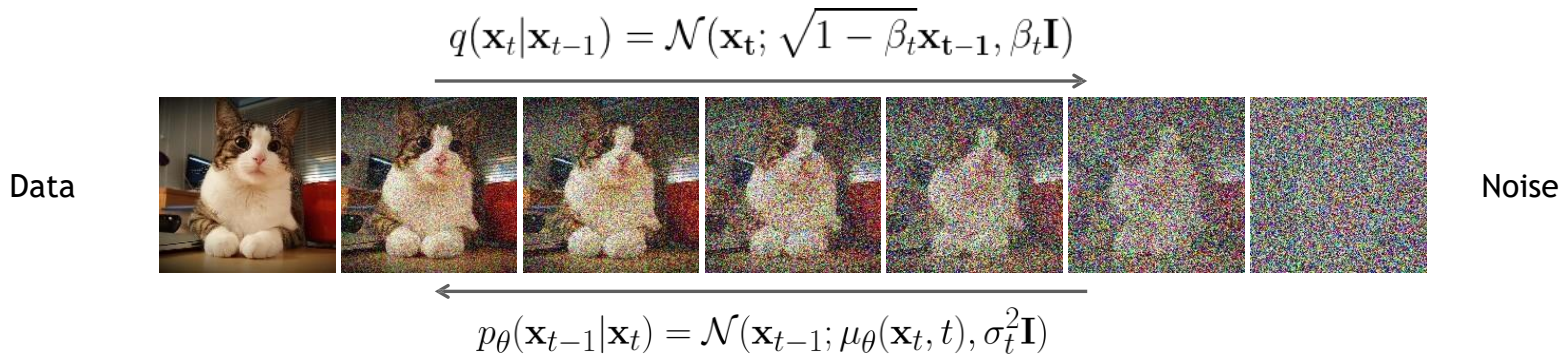


Time representation: sinusoidal positional embeddings or random Fourier features.

Time features are fed to the residual blocks using either simple spatial addition or using adaptive group normalization layers. (see [Dharivwal and Nichol NeurIPS 2021](#))

Diffusion Parameters

Noise Schedule



Above, β_t and σ_t^2 control the variance of the forward diffusion and reverse denoising processes respectively.

Often a linear schedule is used for β_t , and σ_t^2 is set equal to β_t .

[Kingma et al. NeurIPS 2022](#) introduce a new parameterization of diffusion models using signal-to-noise ratio (SNR), and show how to learn the noise schedule by minimizing the variance of the training objective.

We can also train σ_t^2 while training the diffusion model by minimizing the variational bound ([Improved DPM by Nichol and Dhariwal ICML 2021](#)) or after training the diffusion model ([Analytic-DPM by Bao et al. ICLR 2022](#)).

Denoising Diffusion Probabilistic Models



Figure 3: LSUN Church samples. FID=7.89

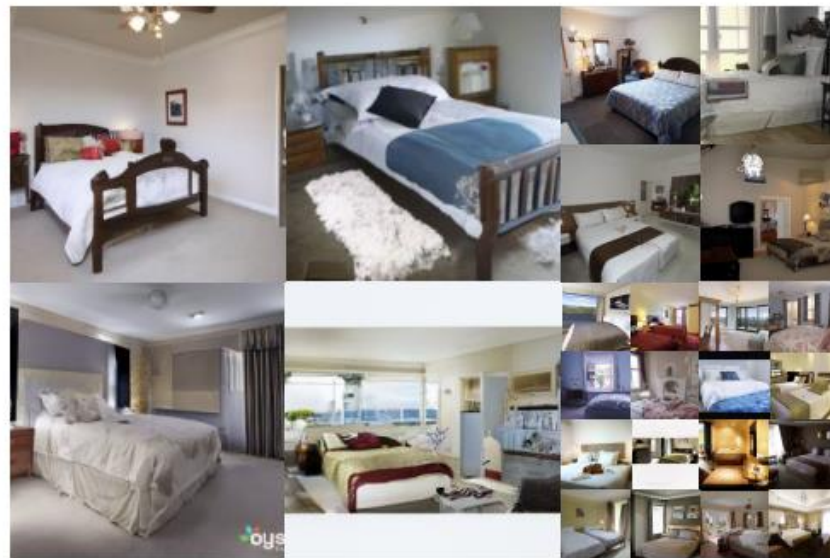


Figure 4: LSUN Bedroom samples. FID=4.90

Case study: Imagen

Imagen: text-to-image diffusion models

By Google (imagen.research.google)

Input: text; Output: 1kx1k images

- An unprecedented degree of photorealism
 - SOTA automatic scores & human ratings
- A deep level of language understanding
- Extremely simple
 - no latent space, no quantization

6
3



A brain riding a rocketship heading towards the moon.

Imagen

By Google (imagen.research.google/)



A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.

Imagen

By Google (imagen.research.google/)



A dragon fruit wearing karate belt in the snow.

"toilet paper with real cactus spikes" *by Irina Blok*



What goes to Imagen?

Data

- Image-text pairs
- LAION-400M
- Internal (~500M images)

Sampler

- Classifier-free guidance
- Maximizing text-alignment

Model

- Diffusion models
- Cascading super-res
- Frozen text encoders

Scaling up

What goes to Imagen?

Data

- Image-text pairs
- LAION-400M
- Internal (~500M images)

Sampler

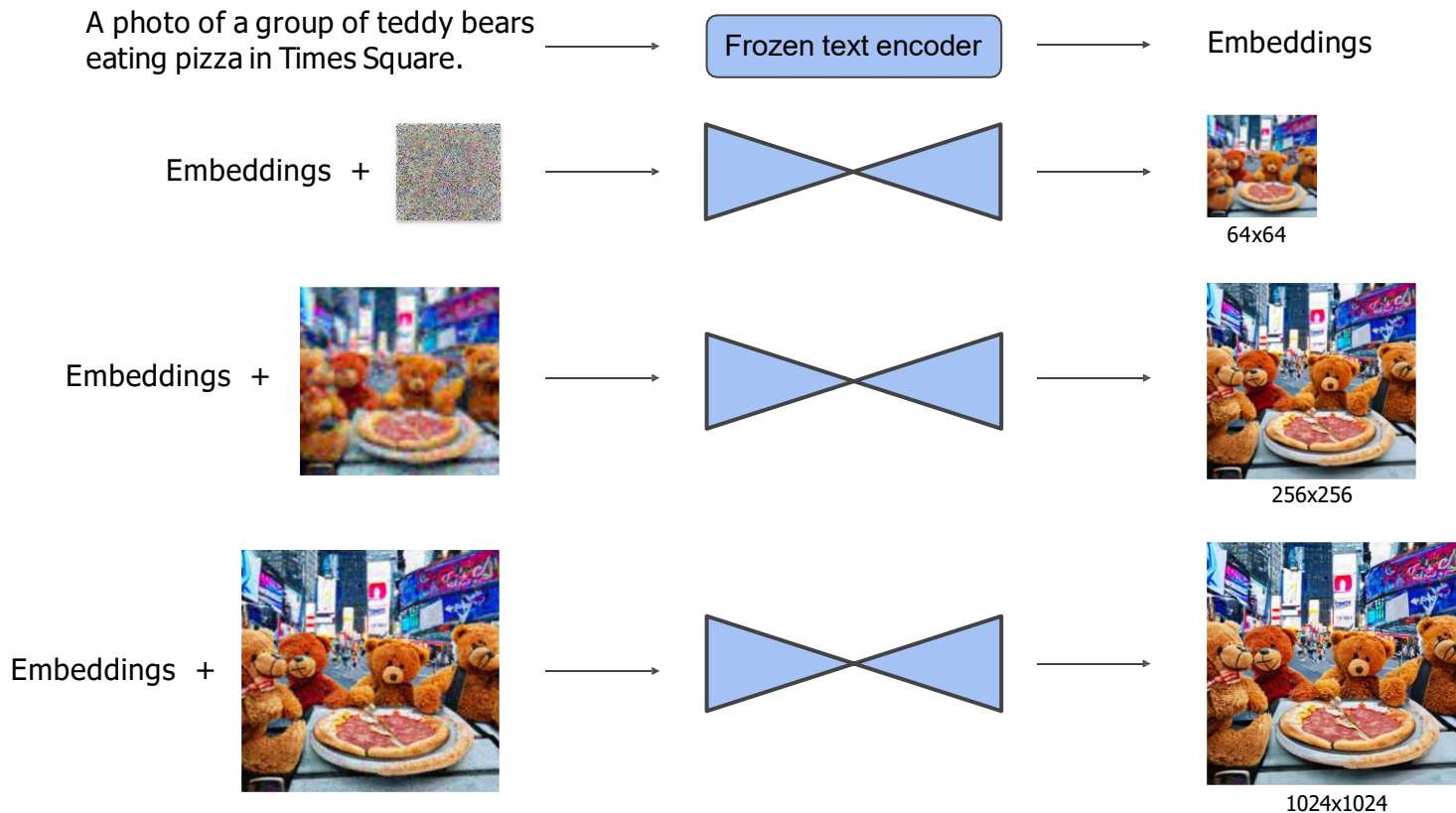
- Classifier-free guidance
- Maximizing text-alignment

Model

- Diffusion models
- Cascading super-res
- Frozen text encoders

Scaling up

Imagen: Cascaded generation pipeline



Classifier Guidance

Sampler technique

- Assume pairs of data $(\mathbf{x}, \mathbf{c})$. A classifier guidance diffusion model consists of
 - A trained conditional diffusion model
 - A trained classifier model on noisy data $\mathbf{x}_t$
- During sampling, at each denoising step, modify the score function to

$$\nabla_{\mathbf{x}_t} \log \tilde{p}_{\theta, \phi}(\mathbf{x}_t | \mathbf{c}) = \nabla_{\mathbf{x}_t} \log p_{\theta}(\mathbf{x}_t | \mathbf{c}) + \omega \nabla_{\mathbf{x}_t} \log p_{\phi}(\mathbf{c} | \mathbf{x}_t).$$

From the conditional
diffusion model

From the classifier
model

- Upweight samples that the classifier assigns high probability with, better alignment with $\mathbf{c}$.
- Cons: need to train an additional classifier. Increase model complexity.

Classifier-free Guidance

Sampler technique

- Assume there're two diffusion models, one conditional model and one unconditional model.
- By Bayes' rule we can define an implicit classifier

$$p_{\theta}(\mathbf{c}|\mathbf{x}_t) \propto p_{\theta}(\mathbf{x}_t|\mathbf{c})/p_{\theta}(\mathbf{x}_t).$$

- The modified score function during sampling then becomes

$$\nabla_{\mathbf{x}_t} \log \tilde{p}_{\theta}(\mathbf{x}_t|\mathbf{c}) = (1 + \omega) \nabla_{\mathbf{x}_t} \log p_{\theta}(\mathbf{x}_t|\mathbf{c}) - \omega \nabla_{\mathbf{x}_t} \log p_{\theta}(\mathbf{x}_t).$$

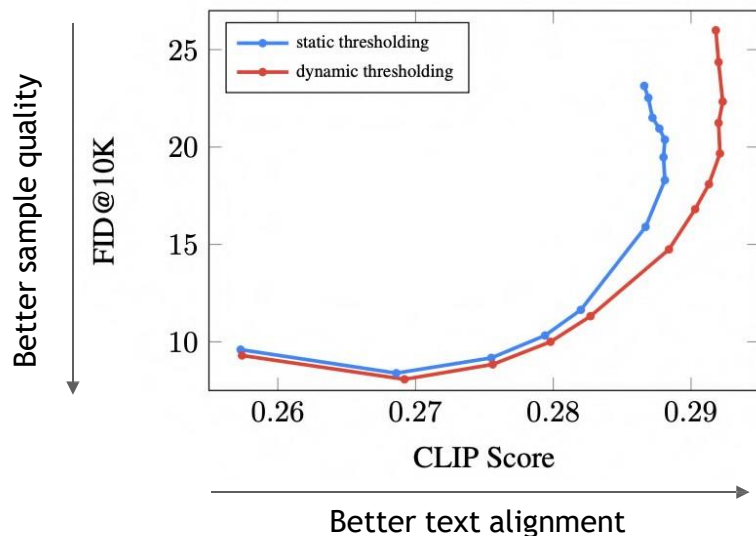
From the conditional
diffusion model

From the unconditional
diffusion model

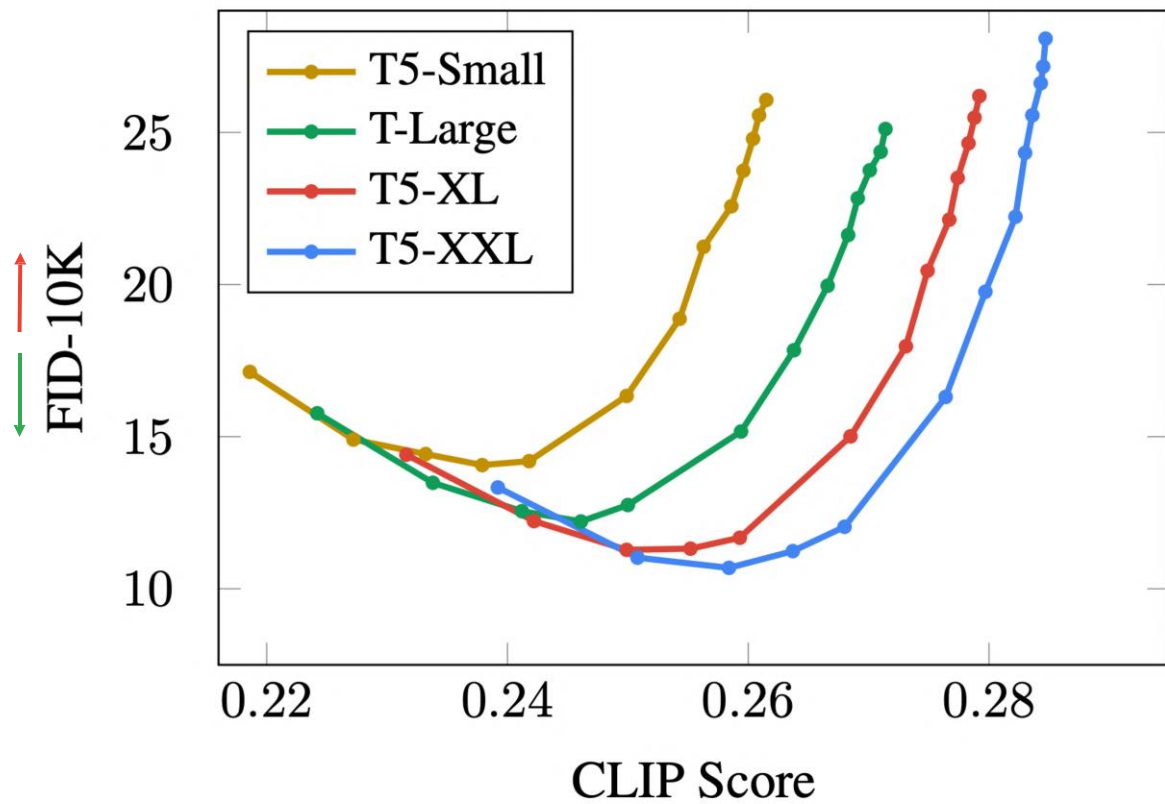
- The two models can share weights, with the unconditional model taking a null class label c .

Classifier-free guidance in Imagen

- Large classifier-free guidance weights → better text alignment, worse image fidelity



Larger Text Encoders → Better Alignment, Better Fidelity

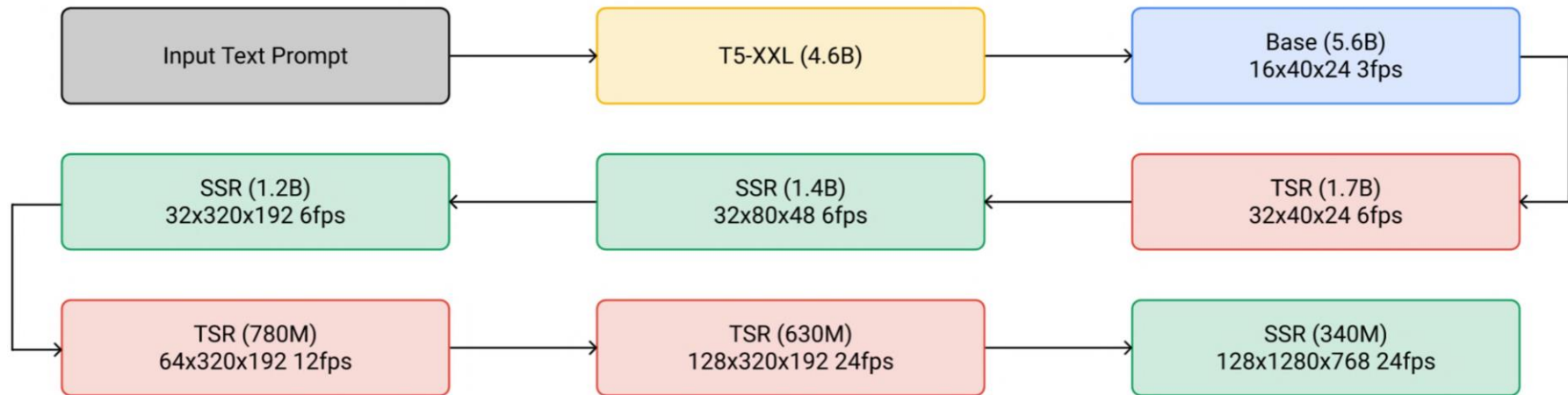


Can we generalize this to video?

Imagen Video (imagen.research.google/video)



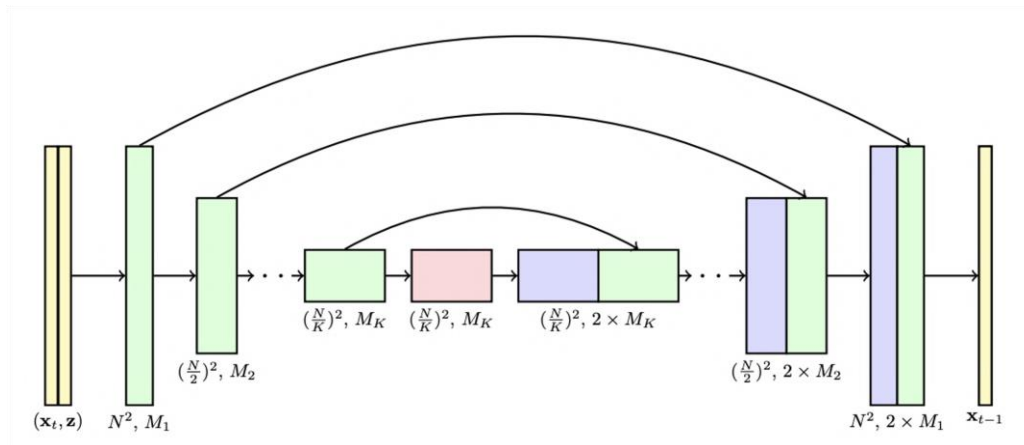
Imagen Video generalizes Imagen to the video domain using a cascade of super-resolution diffusion models in space and time



Video diffusion models

Ho & Salimans, et al. <https://video-diffusion.github.io/>

- Image $\rightarrow$ Video: Just add another dimension to the data tensor
- Image architecture: 2D UNet
- Video architecture: 3D UNet, space-time separable
 - repeat the 2D UNet over frames
 - additional layers to mix over time using attention or convolution





Wooden figurine surfing on a surfboard in space.





Flying through an intense battle between pirate ships in a stormy ocean.

Imagen Video

*“wateícoloí gíeeting caíd of
thank you so much!”*

